

Excelerate Data Analysis

Opportunity Data & Learner Data — Exploratory Data Analysis (EDA) Report

PROJECT OVERVIEW AND EXECUTIVE SUMMARY

This report documents the Exploratory Data Analysis (EDA) performed on two primary datasets imported into PostgreSQL from Excelerate: the **Opportunity Data** and **Learner Data** tables. The initial goal was to evaluate the structure, quality, and completeness of the datasets ahead of the cleaning and integration work planned for Week 2, rather than drawing final analytical conclusions.

However, the initial audit reveals severe data gaps and structural anomalies that currently paralyze analytical capabilities. The most critical finding is a massive tracking failure within the Learner data table, where 13,372 records are missing an `'application_id'`. This omission leaves the majority of the dataset unidentifiable, rendering it impossible to perform de-duplication checks or reliably link learners to the 5,733 records in the Opportunity Dataset.

Furthermore, operational analytics are completely locked due to type conversion failures during import, which forced all destination fields into a flat text format. Proceeding with performance reporting under these current conditions will result in highly skewed metrics and artificially inflated volumes. Therefore, immediate technical remediation is required to isolate these text anomalies, properly cast fields into their respective datatypes, and address the tracking failures before further integration and analysis can proceed.

DATABASE SETUP

To load the Learner Dataset and Opportunity Dataset to PostgreSQL here are the steps that were taken:

1. The data was downloaded from the Excelerate Learning Management System(LMS) as a .csv file.
2. Data was inspected using Excel Spreadsheets and Notepad. It was discovered that there were anomalies that would impact the import process into PostgreSQL.
3. Due to a high volume of unnamed and contiguous columns in the source files, temporary text-based staging tables were provisioned. This allowed for an uninterrupted, bulk copy of the data without risk of schema mismatch errors.
4. Transformation scripts, were executed to parse, name, and isolate individual data elements from the staging block using commas as the delimiter and assigning index positions to destination fields.
5. For the time being, all columns are in the text format and attempts to convert failed due to the mixed data types in the fields.

Here is a snippet of the data:

pk text	sk text	assigned_cohort text	fee text	status text	accept_reject_da text
Learner#	Learner#0000000010Z56C78JF7M6VQXQ2	[null]	0	[null]	[null]
Learner#838ff3db-f3de-4ae4-9c06-29632039ad...	Learner#000000000G1RYRMC22H9RYCY...	[null]	[null]	1010	[null]
Learner#	Learner#0000000010K1EA93ZGXZA0S5NG	[null]	33	[null]	[null]
Learner#e84aea65-34d3-440a-bf04-f877d3bc5c...	Learner#0000000010MXXW8GCFYE5S30...	BFSHUCM	[null]	1070	1697094086113
Learner#f9f52ded-a970-4700-830a-c13df19749...	Learner#0000000010BMP2H74SMG8HW...	BU5NLYK	[null]	1070	1774502025544
Learner#8d4c12bb-211d-459f-8b2d-653095f9b5...	Learner#0000000010V1FQTM6225F4HG8F	[null]	[null]	1030	1714217388011
Learner#6f39a383-aa51-4642-8995-61e49649f6...	Learner#00000000104G9QFHECKMCD2A...	[null]	[null]	1010	[null]
Learner#c577f426-abe7-4f04-a8db-92cff7ca86bd	Learner#000000000G1RYRMC22H9RYCY...	[null]	[null]	1030	1682333589131
Learner#e84aea65-34d3-440a-bf04-f877d3bc5c...	Learner#0000000010CTBSXC407YQRNBXZ	B89L7YP	[null]	1070	1697088776119
Learner#6ca14518-2878-4135-adc1-06e6f0605...	Learner#0000000010VM9E97B0CBPMNZ...	B3V0FWZ	[null]	1120	1721382458168

Fig 1: Learner Data

pk text	opportunity_id text	Badge text
Opportunit...	Opportunity#000000000GBD0AX3Z6VYRG7R...	{'sk': 'Badge#000000000G87VEXNYKKKTJKE9', 'ref_properties': {'\relation_id\': 'K9MSIZ\'}}}
Opportunit...	Opportunity#000000000GFKZ4N696ZF8V2W...	{'sk': 'Badge#000000000GAR6YSZ2B71HZJPM3', 'ref_properties': {'\relation_id\': '6AJ5FD\'}}}
Opportunit...	Opportunity#0000000010ZFE0ZS22FA3Q62R4	[null]
Opportunit...	Opportunity#000000000GG34EPWZAJ6BDP...	{'sk': 'Badge#000000000GGQ61DZ2TSSHR55G9', 'ref_properties': {'\relation_id\': '76DV7Q\'}}}
Opportunit...	Opportunity#0000000010ZNX2EXET8MGC9F...	[null]
Opportunit...	Opportunity#000000000GQHDFWJ9R1HJQJ4...	[null]
Opportunit...	Opportunity#000000000GQK8HHSCDYA3T...	{'sk': 'Badge#000000000G5A7GFSWAQC3BGV6', 'ref_properties': {'\relation_id\': '5DTUK1\'}}, {'sk': 'Badge#0000
Opportunit...	Opportunity#0000000010ZK125W68Z6BG2E...	{'sk': 'Badge#00000000106BCP0TFB1Y2D0V0Q', 'created_at': '1727707449137', 'ref_properties': {'\relation_id\': 'N
Opportunit...	Opportunity#000000000GRHVKG2HQQFC6...	{'sk': 'Badge#000000000G1X85MGR4KRAZJYK8', 'ref_properties': {'\test\': 'test\'}}, {'\relation_id\': '9GL03J\'}}}
Opportunit...	Opportunity#0000000010YHJCSGAKECPBFQ...	[null]

Fig 2: Opportunities Data

DATASET OVERVIEW

Opportunity Data Exploration

Exploration of the initial dataset shows that there are a total of 5733 records and 33 fields. The Opportunity Data dataset contains information about opportunities offered through Excelerate. The following queries were used to explore its structure and content.

1. Raw structure preview

A simple SELECT * was run first to inspect the raw column structure and confirm the data loaded as expected.

```
SELECT * FROM public.opportunity_data;
```

The screenshot shows the pgAdmin 4 interface with a query window displaying the results of the query `SELECT * FROM public.opportunity_data`. The results are shown in a table with columns: pk, opportunity_id, and badge. The table contains 8 rows of data, with the first row being the header and the subsequent rows containing opportunity details. The status bar at the bottom indicates 'Total rows: 5734' and 'Query complete 00:00:00.763'.

pk	opportunity_id	badge
1	pk	text
2	Opportunity#	Opportunity#0000000000G800AX3Z6VYRG7R...
3	Opportunity#	Opportunity#0000000000GFKZ4N695ZF8V2W...
4	Opportunity#	Opportunity#0000000000DZFED232FA3Q63R4...
5	Opportunity#	Opportunity#0000000000GG34EPWZAJ6BDP...
6	Opportunity#	Opportunity#000000000010W2DE28TMPKJR...
7	Opportunity#	Opportunity#0000000000GQHDFWJ9R1HQJ4...
8	Opportunity#	Opportunity#0000000000GQK8HHSDDCA3T...

2. What categories of opportunities are available?

```
SELECT category, COUNT(*) AS total
FROM opportunity_data
GROUP BY category
ORDER BY total DESC;
```

The screenshot shows the pgAdmin 4 interface with a query window displaying the results of the query `SELECT category, COUNT(*) AS total FROM opportunity_data GROUP BY category ORDER BY total DESC;`. The results are shown in a table with columns: category and total. The table contains 9 rows of data, with the first row being the header and the subsequent rows containing category names and their corresponding counts. The status bar at the bottom indicates 'Total rows: 15' and 'Query complete 00:00:00.283'.

category	total
1	Internship
2	Career
3	Competition
4	Course
5	Event
6	Engagement
7	Masterclass
8	JobSimulation
9	Program

Findings: Internship (1,591) and Career (1,088) are by far the two largest categories, followed by Competition (734), Course (632), and Event (622). A small number of rows (2) fell under a category value of NULL, and a few rows contained corrupted/malformed values (e.g. stray JSON fragments such as "%22value%22: %221%22") rather than a real category name — a data quality issue to flag for cleaning in Week 2.

3. How are rewards distributed across opportunities?

Since the reward column stores messy JSON-like text rather than a clean structured value, the first step was to check how many opportunities have any reward populated at all.

```

SELECT
    CASE WHEN reward IS NULL OR reward = '' THEN 'No Reward' ELSE 'Has Reward' END
AS reward_status,
    COUNT(*)
FROM opportunity_data
GROUP BY reward_status;

```

The screenshot shows the pgAdmin 4 interface. The SQL query is executed, and the results are displayed in the Data Output pane. The query counts the number of opportunities with and without a reward.

reward_status	count
Has Reward	5733
No Reward	1

Successfully run. Total query runtime: 195 msec. 2 rows affected.

5,733 of 5,734 opportunities have a reward value populated; only 1 does not.

This confirmed that reward is almost universally populated, so the more useful next step was breaking reward presence down by category:

```

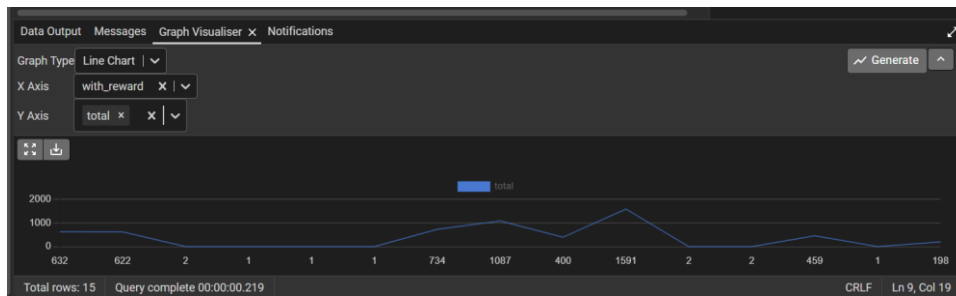
SELECT category, COUNT(*) AS total,
    COUNT(CASE WHEN reward IS NOT NULL AND reward != '' THEN 1 END) AS
with_reward
FROM opportunity_data
GROUP BY category;

```

The screenshot shows the pgAdmin 4 interface. The SQL query is executed, and the results are displayed in the Data Output pane. The query counts the total number of opportunities and the number of opportunities with a reward, grouped by category.

category	total	with_reward
Course	632	632
Event	622	622
NULL	2	2
category	1	1
{ %22label%22: %22United Kingdom%22	1	1
%22value%22: %221%22}	1	1
Competition	734	734
Career	1088	1087
Masterclass	400	400

Successfully run. Total query runtime: 219 msec. 15 rows affected.



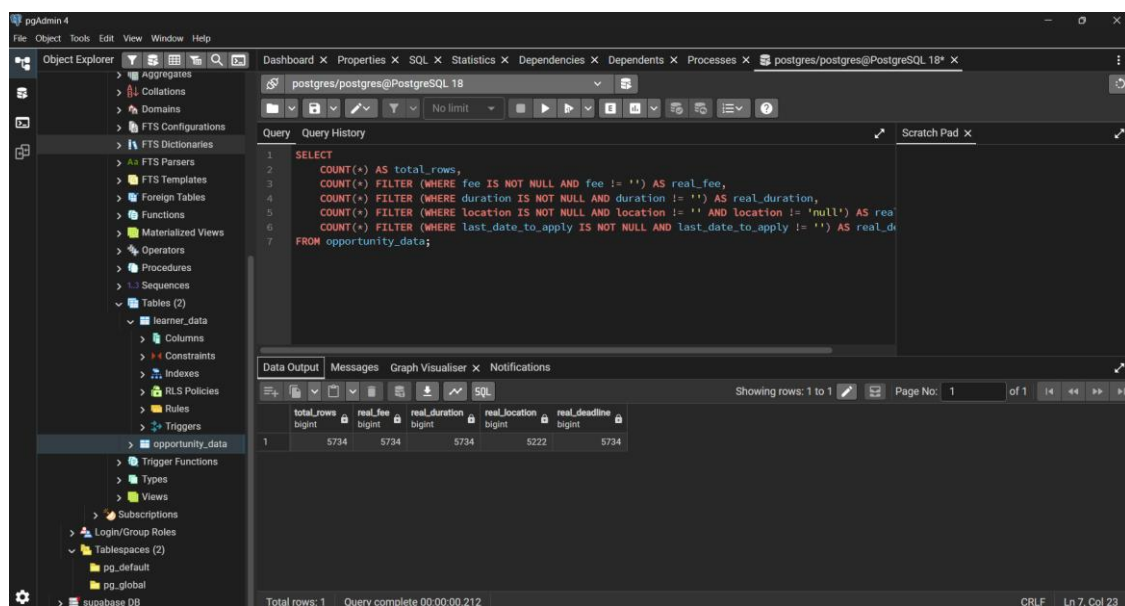
Line chart view of with_reward vs. total (Graph Visualiser).

Findings: reward is populated for virtually every opportunity in every category (e.g. Internship: 1,591/1,591; Career: 1,087/1,088). This means the reward field, once parsed out of its JSON-like text format, could be a genuinely useful field for future performance analysis — but it will need to be cleaned/parsed before it can be used numerically or categorically.

4. Which fields may be useful for performance analysis? (completeness check)

To judge which fields are reliable enough to build analysis on, a completeness check was run comparing total row count against non-blank values for several candidate fields.

```
SELECT
    COUNT(*) FILTER (WHERE fee IS NOT NULL AND fee != '') AS real_fee,
    COUNT(*) FILTER (WHERE duration IS NOT NULL AND duration != '') AS
real_duration,
    COUNT(*) FILTER (WHERE location IS NOT NULL AND location != ''
                        AND location != 'null') AS real_location,
    COUNT(*) FILTER (WHERE last_date_to_apply IS NOT NULL
                        AND last_date_to_apply != '') AS real_deadline,
    COUNT(*) AS total_rows
FROM opportunity_data;
```



Completeness check query and results — fee, duration, and last_date_to_apply are fully populated; location is missing 512 values.

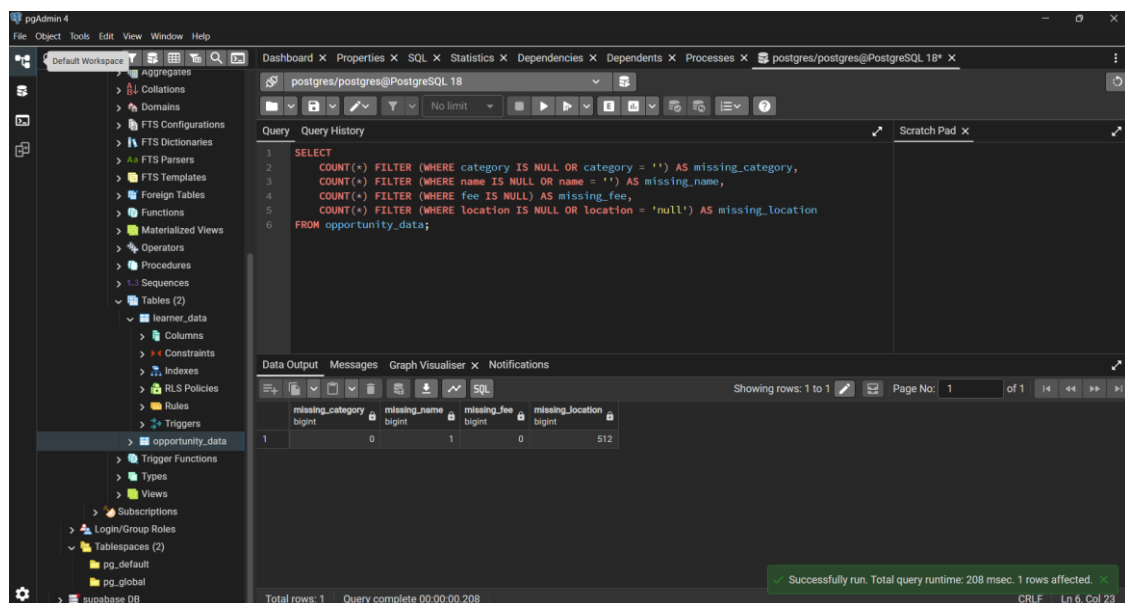
Field	Total Rows	Real (non-blank)	Missing
fee	5,734	5,734	0
duration	5,734	5,734	0
location	5,734	5,222	512
last_date_to_apply (deadline)	5,734	5,734	0

Summary of field completeness across the four candidate fields.

Findings: fee, duration, and last_date_to_apply are fully populated across every one of the 5,734 records, making them reliable candidates for performance analysis (e.g. cost trends, program length, application deadline timing). location is missing in 512 rows (~9%) and will need a decision in Week 2 on how to handle those gaps (exclude, or label as "Unknown").

5. Are there any missing or inconsistent records?

```
SELECT
  COUNT(*) FILTER (WHERE category IS NULL OR category = '') AS missing_category,
  COUNT(*) FILTER (WHERE name IS NULL OR name = '') AS missing_name,
  COUNT(*) FILTER (WHERE fee IS NULL) AS missing_fee,
  COUNT(*) FILTER (WHERE location IS NULL OR location = 'null') AS
missing_location
FROM opportunity_data;
```



The screenshot shows the pgAdmin 4 interface. The SQL query is entered in the Query tab, and the results are displayed in the Data Output tab. The query counts missing values for category, name, fee, and location. The results show 0 missing values for category, 1 for name, 0 for fee, and 512 for location.

missing_category	missing_name	missing_fee	missing_location
0	1	0	512

Missing-value counts for category, name, fee, and location.

Findings: category is fully populated (0 missing), name has just 1 missing row, fee has 0 missing, and location again shows 512 missing values — consistent with the completeness check above. Overall the dataset is highly complete, with location being the one field that will need explicit handling before it can be used reliably.

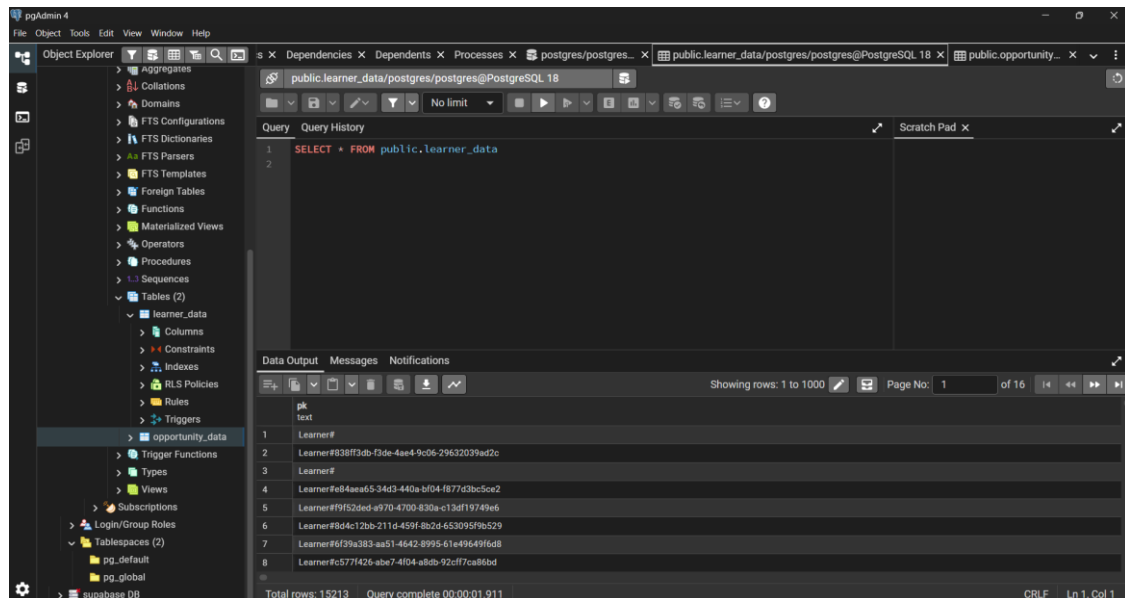
Learner Data Exploration

Initial exploration of the Learner Data shows that there are a total of 15029 records and 43 columns.

The Learner Data dataset captures learner activity and engagement — applications, sign-ups, completions, and related metadata.

1. Raw structure preview

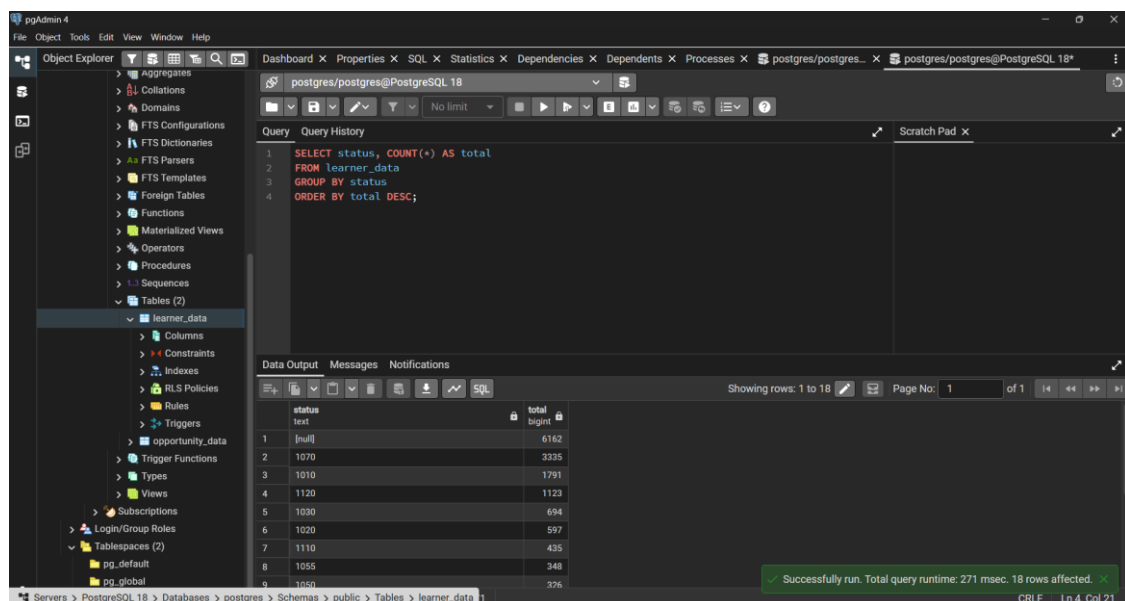
```
SELECT * FROM public.learner_data;
```



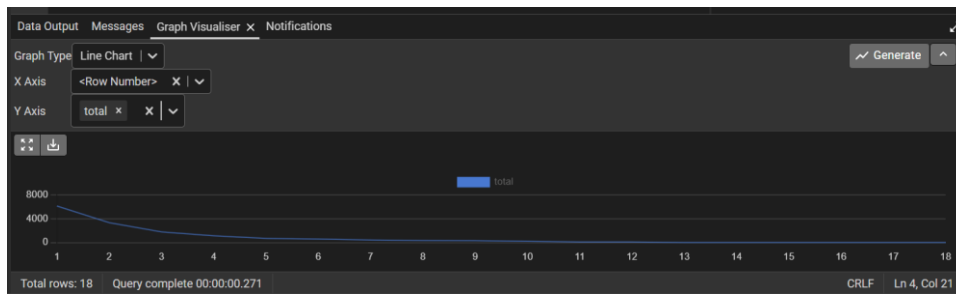
Sample rows from learner_data (15,213 total rows), showing the pk column with Learner# identifiers.

2. Application records (status distribution)

```
SELECT status, COUNT(*) AS total
FROM learner_data
GROUP BY status
ORDER BY total DESC;
```

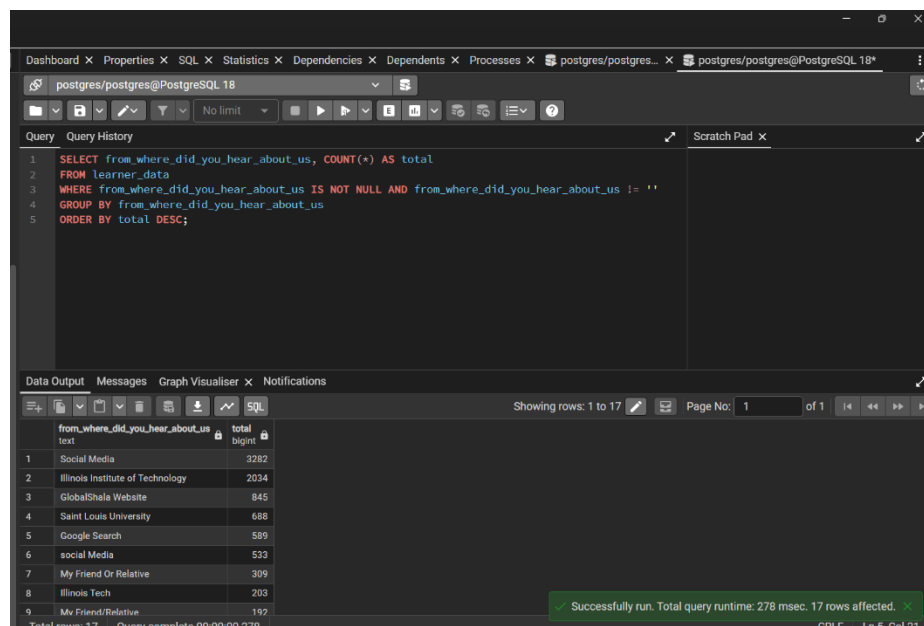
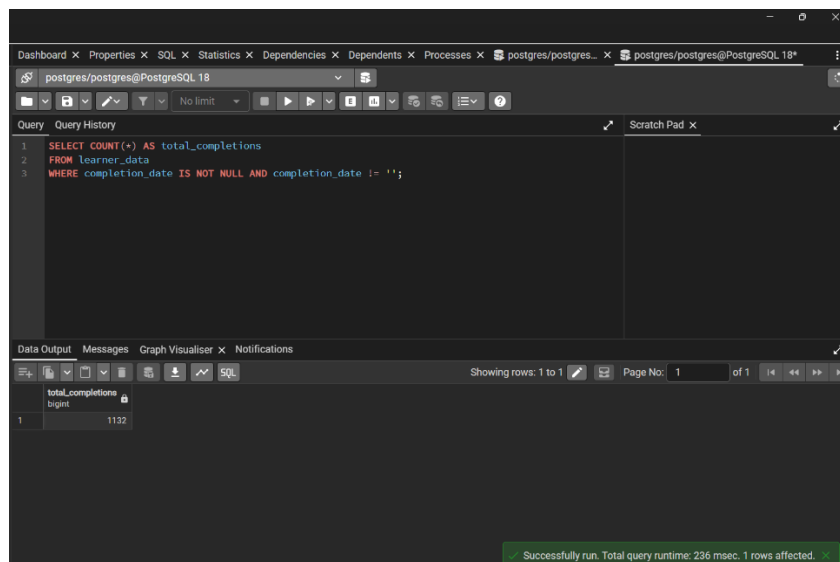


Application status distribution across all learner records.



Line chart view of status distribution, showing NULL and code 1070 as the dominant groups.

Findings: the largest single group (6,162 records) has a NULL status, which is the single biggest data quality issue found in either dataset so far — over 40% of learner records have no status recorded. Among records with a status, the codes 1070 (3,335), 1010 (1,791), and 1120 (1,123) are the most common, though the meaning of these numeric status codes is not yet documented and will need to be mapped to readable labels (e.g. Applied, Accepted, Completed, Withdrawn) in Week 2.



Lead sources-where it was discovered.

Dashboard X Properties X SQL X Statistics X Dependencies X Dependents X Processes X postgres/postgres... X postgres/postgres@PostgreSQL 18*

postgres/postgres@PostgreSQL 18

Query Query History Scratch Pad

```

1 SELECT payment_status, COUNT(*) AS total,
2     AVG(amount_to_be_paid::numeric) AS avg_amount
3 FROM learner_data
4 WHERE amount_to_be_paid ~ '^[0-9]+(\.[0-9]+)?$'
5 GROUP BY payment_status;

```

Data Output Messages Graph Visualiser X Notifications

Showing rows: 1 to 3 Page No: 1 of 1

	payment_status	total	avg_amount
	text	bigint	numeric
1	[null]	13	0.00000000000000000000
2	paid	504	189.7023809523809524
3	unpaid	8518	417.2347969006809110

Successfully run. Total query runtime: 240 msec. 3 rows affected.

Total rows: 3 Query complete 00:00:00.240 CRLF Ln 5, Col 25

EXPLORATORY ANALYSIS

1. Opportunities by category

Internships, career opportunities, and competitions dominate the catalogue; Program and Xploreu are effectively pilot categories with only 2 opportunities each.

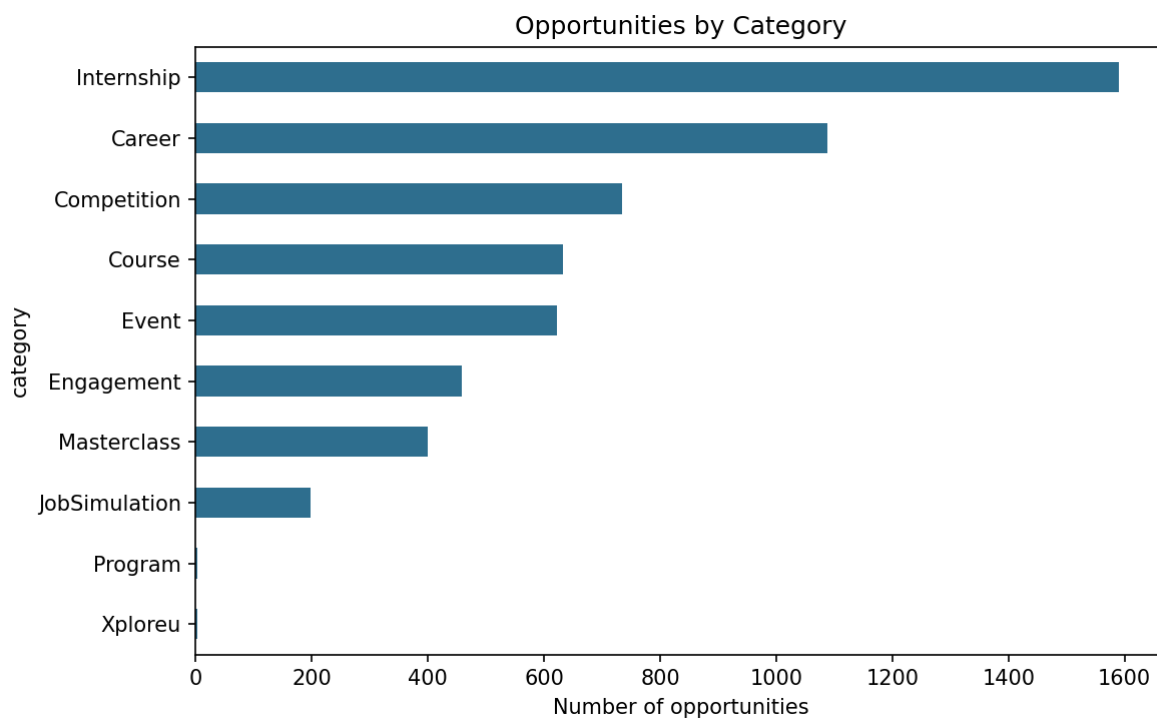


Figure 1 — Count of opportunities per category (opportunities table, corrupted category values excluded).

2. Application activity over time

Applications (apply_date on the learner side) span mid-2022 through early-2026, with visible month-to-month volume swings that likely track cohort launch dates and marketing campaigns rather than steady organic growth.

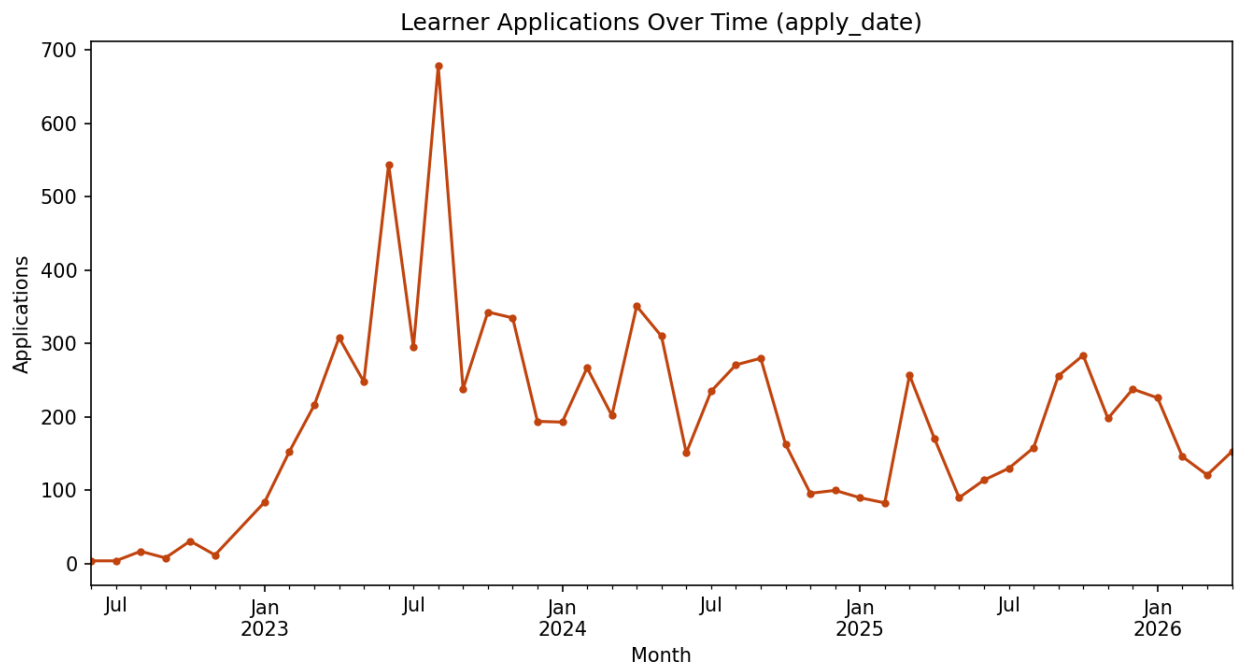


Figure 2 — Monthly count of learner applications (apply_date), valid learner rows only.

3. Application status funnel

Team Allocated (status 1070) is the single largest status among valid rows, followed by Applied (1010) and Rewards (1120). Only 597 rows are marked Accepted (1020) against 694 Rejected (1030), and 435 Withdraw (1110) — worth validating against the business's expected funnel shape.

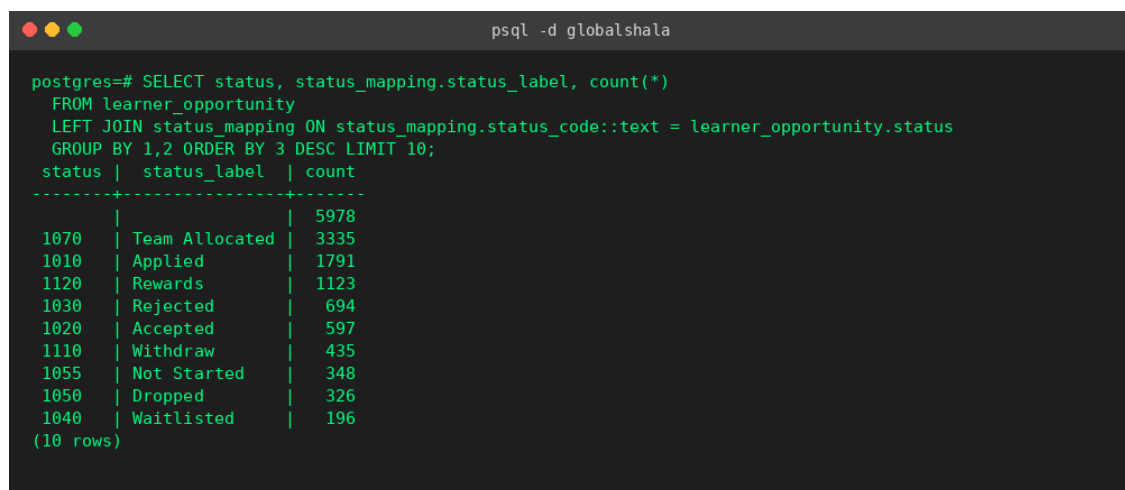


Figure 3 — Top status codes joined against status_mapping, run directly against the PostgreSQL database.

4. Missing data pattern

Columns fall into three clear bands: near-complete identifiers (pk, created_at), “core application” fields present on roughly 60% of rows (status, fee, category, apply_date — i.e. present only on the valid learner rows), and long-tail free-text/consent fields answered by a small fraction of applicants (e.g. “please_identify_your_student_status” at 6 responses).



Figure 4 — Percentage missing per named column, learner_opportunity.

5. What else the data supports

- Channel attribution analysis via from_where_did_you_hear_about_us (Social Media, university partners, Google Search, referrals) — once free-text values are normalised (e.g. “Social Media” vs “social Media”).
- Cohort-level analysis via assigned_cohort / cohort_p1 / cohort_p2 short codes, once their relationship to a specific opportunity or cohort record is restored.
- Time-to-decision and drop-off analysis using apply_date, accept_reject_date, not_started_date, dropped_out_date, completion_date, reward_awarded_date.
- Opportunity catalogue analysis (fee/duration/category mix, eligibility, panel/testimonial coverage) once the corrupted rows are removed from opportunities.

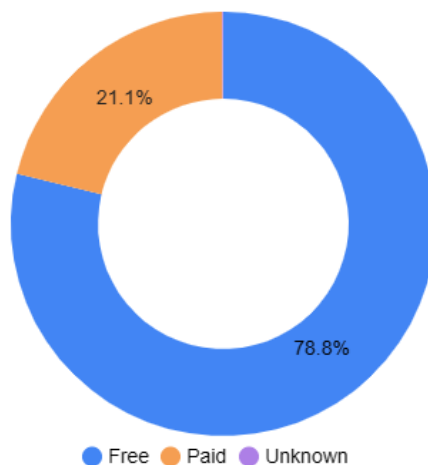
6. Notable distributions

```
WITH cleaned_fee AS (  
  SELECT  
    CASE  
      WHEN TRIM(fee) IN ('', 'NULL') OR fee IS NULL  
      THEN NULL  
      ELSE TRIM(fee)::NUMERIC  
    END AS fee_value  
  FROM opportunity  
  WHERE pk = 'Opportunity#'  
)  
  
SELECT  
  CASE  
    WHEN fee_value = 0 THEN 'Free'  
    WHEN fee_value > 0 THEN 'Paid'  
    ELSE 'Unknown'  
  END AS fee_type,  
  COUNT(*) AS total_opportunities,  
  ROUND(  
    COUNT(*) * 100.0 / SUM(COUNT(*)) OVER (),  
    2  
  ) AS percentage  
FROM cleaned_fee  
GROUP BY fee_type  
ORDER BY total_opportunities DESC;
```

Opportunities by Fee

Fee Type	Total Opportunities	Percentage
Free	4518	78.85
Paid	1210	21.12
Unknown	2	0.03

Fee Type Distribution



Opportunities by Fee Type Distribution

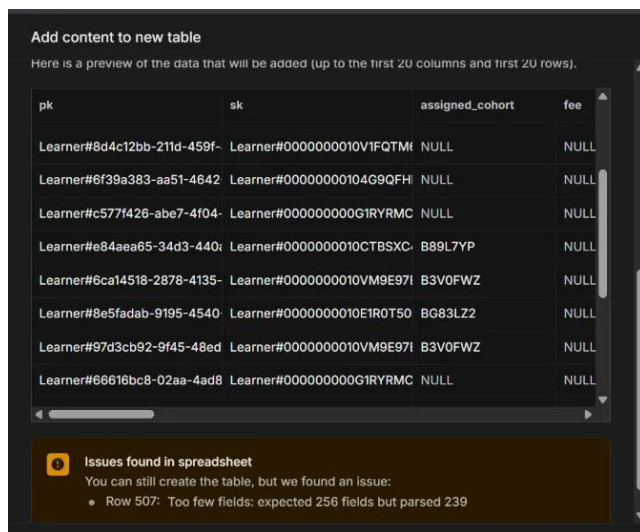
The data presented in reveals a significant skew toward non-monetized offerings, with "Free" opportunities heavily dominating the distribution. Specifically, free offerings account for 78.85% (4,518) of the total opportunities, while "Paid" options make up a much smaller secondary segment at exactly 21.12% (1,210). Furthermore, the dataset exhibits excellent completeness regarding fee structures, as a negligible fraction—just two opportunities, or 0.03%—fall into the "Unknown"

category. Overall, this distribution highlights an ecosystem or platform that relies predominantly on free access over paid transactions.

DATA QUALITY ASSESSMENT

1. Missing / Structural Issues

- Malformed rows: Row 463 of the Learner Data export parsed with far fewer fields than expected (88 vs. 306), and a similar issue was observed at Row 507 in an earlier pass — indicating inconsistent field counts in the source export that could shift column alignment if not corrected.
- Empty trailing columns: Roughly 260 of the original 306 columns in the raw Learner Data file were entirely empty and unnamed (_1 through _261), consistent with an export artifact rather than real data.
- NoSQL-style structure: The presence of pk / sk fields (partition key / sort key) suggests the source system is non-relational, which may explain the sparsity and inconsistency in the raw export.



'Issues found in spreadsheet' — Row 507 error

2. Duplicate records

Learners Duplicate by PK: **304** rows

Opportunity Duplicate by Opportunity_ID: **0** rows

3. Inconsistent formatting

marketing_source	total_records
Social Media	3282
Illinois Institute of Technology	2034
GlobalShala Website	845
Saint Louis University	688
Google Search	589
social Media	533

My Friend Or Relative	309
Illinois Tech	203
My Friend/Relative	192
Northeastern University	128
Global Enrichment Program	46
Others	38
Other	16
Rochester Institute of Technology	8
%22key%22: %22country%22	1
SLU	1
Hiring Platforms	1

4. Unusual or unexpected values

opportunity_id	name	category	fee	currency_type
Opportunity#000000001043VZ...	Testing auto approve	Career	6000000000000.00	USD
Opportunity#0000000010Y14...	financial skills development	Career	200000.00	USD
Opportunity#0000000010DYC...	FULL STACK WEB DEV	Internship	65000.00	INR
Opportunity#00000000105ZX5...	testing event sl	Course	20000.00	USD
Opportunity#0000000010NH1...	yfughehg	Internship	2345.00	USD
Opportunity#000000000G1RY...	Cybersecurity: Defensive Hack	Internship	2000.00	USD
Opportunity#00000000104XK...	VISHWAJEET	Internship	2000.00	USD
Opportunity#0000000010PAD...	Cybersecurity: Defensive Hack	Internship	2000.00	USD
Opportunity#00000000101ASJ...	Opportunity name :234234	Internship	1234.00	USD
Opportunity#0000000010EAT...	Normal	Internship	1000.00	USD

5. Explain how these issues may affect future analysis.

Data Quality Issue	Possible Effect on Analysis
Missing values	May reduce the number of usable records and produce incomplete results
Duplicate records	Can inflate application counts, totals, averages, and percentages
Inconsistent names	Splits one channel into multiple categories, causing incorrect analysis
Extreme fee values	Can heavily distort averages and visualizations
Malformed/non-learner rows	Can substantially inflate learner/application counts and may cause SQL conversion errors
NULL, Null, null stored as text	Looker Studio or SQL may treat them as real categories rather than missing data

CONCLUSION & NEXT STEPS

The exploratory data analysis of the Excelerate Opportunity and Learner datasets reveals that while the data holds strong potential for tracking performance, severe structural anomalies and data gaps currently paralyze analytical capabilities. Proceeding with performance reporting under the current conditions would result in highly skewed metrics and artificially inflated volumes.

To successfully unlock the dataset for operational analytics in Week 2, immediate technical remediation must prioritize several critical roadblocks:

Datatype Casting: All destination fields are currently locked in a flat text format due to type conversion failures during the bulk import, which must be properly parsed and cast into correct datatypes.

Tracking Failures: A massive gap exists in the Learner dataset where 13,372 records are missing an 'application_id', making it impossible to perform de-duplication or reliably link learners to the 5,733 Opportunity records.

Missing Status Codes: Over 40% of the learner records (6,162 rows) contain a NULL application status, representing the largest single data quality issue in the datasets.

Data Integrity & Formatting: Inconsistent text formatting (e.g., multiple capitalizations of "Social Media"), JSON-like text fragments stored in categorical fields, and extreme numerical outliers (such as test rows with a fee of 6,000,000,000,000.00 USD) must be standardized or removed to prevent distorted visualizations and averages.

Despite these structural challenges, there is a solid foundation to build upon. The Opportunity dataset features highly complete and reliable data for 'fee', 'duration', and 'last_date_to_apply' across all 5,734 records. Once the text anomalies are isolated and the data is cleaned, these datasets will be well-positioned to support robust cohort tracking, channel attribution, and time-to-decision funnel analysis.